QCon
全球软件开发大会

InfoQ 技术大会

别再给 Agent 一个「毛坯房」了

燕五–蚂蚁集团–CIO技术部

构建 Agent 拎包入住的环境实践



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 走到哪一步

02 编码不是瓶颈

03 Agent住哪里

04 Agent友好环境

05 最大的挑战

06 总结&未来规划

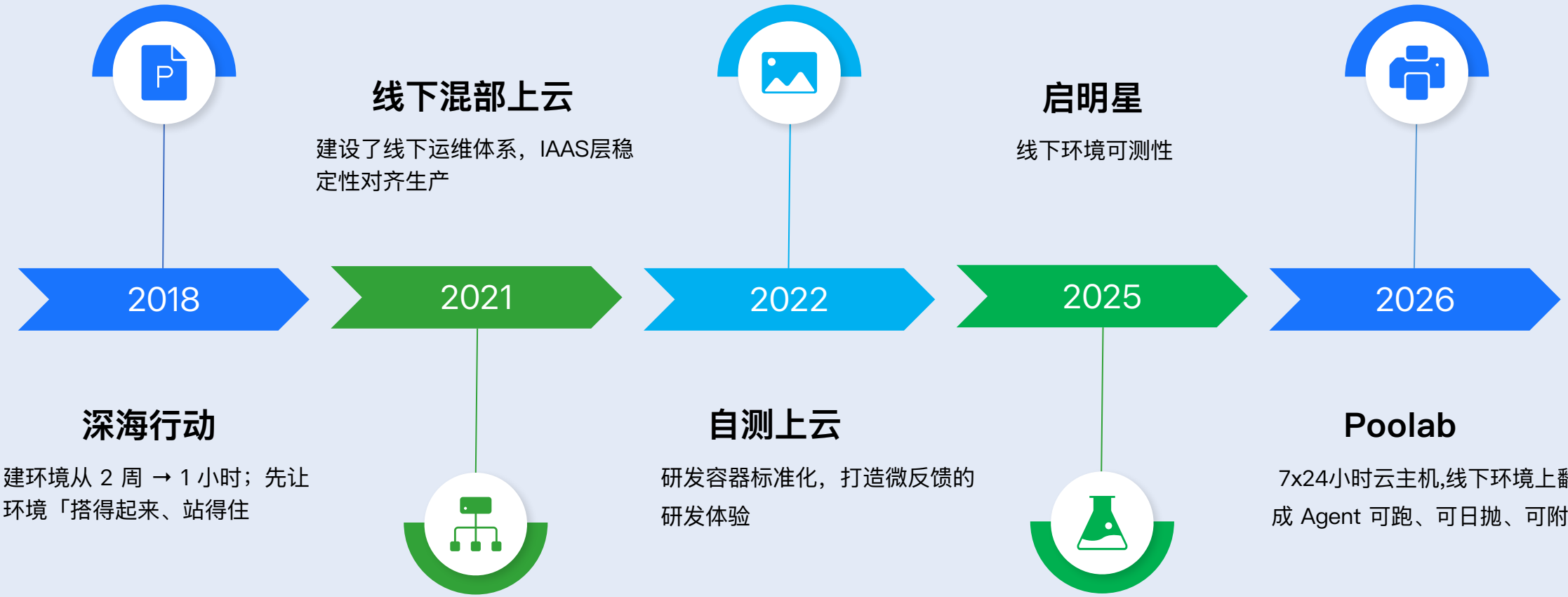
01

走到哪一步

前三阶段给人用，第四阶段换成 Agent

蚂蚁线下环境的发展

工具 → 解决方案 → 智能化 → Agent化



02

编码不是瓶颈

生码再快，整链上限仍有限

环境才是瓶颈


编码不是瓶颈，环境才是
真实案例：编码 1 小时，上线 3 周 • 编码只占整条研发链路 <1%
1
编码 + 本地验证
1 小时
<1%
2
跨平台影响分析/方案评审
1–2 天
3
联调环境准备
2–3 天
4
多平台联调
5–7 天
5
发布审批与编排
3–4 天
6
灰度验证与观察
2–3 天
7
封网等待与合规检查
7–10 天
合计约 3 周（~21 天）
优化的如果不是瓶颈，就不可能带来数量级的效率提升。真正的效率杠杆在环境、联调、发布与验证。

环境工程



任一端为0，结果为0

贬值区

- 微调模型
- 围提示词技巧
- 精细编排生码流程
- 被下一代模型吸收

增值区

- 存量翻新：让 Agent 住进环境
- 环境内验证：提升反馈回路
- 共享文件：多 Agent 协同
- 模型越强越值钱（复利）

03

Agent住哪里

核心不是多几个 CLI，是住进环境

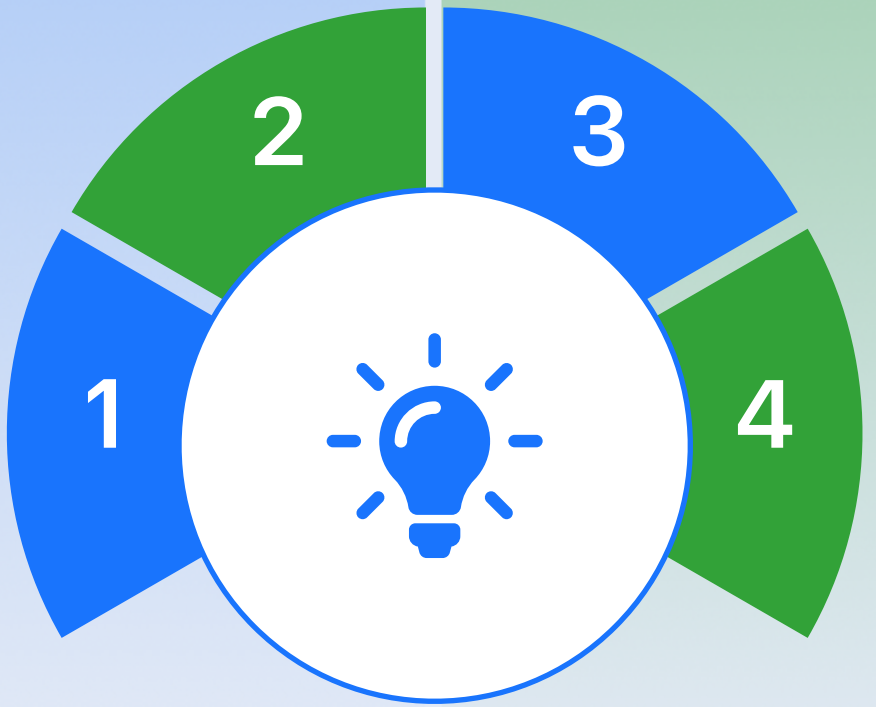
本地的问题

安全隔离

Agent 直接跑在你的电脑上，rm –rf 就删你的真实文件

随时会中断

你合上电脑，Agent 就停了



弹性并发

你的笔记本跑不了 100 个 Agent 并行

标准化

做不到开箱即用

四大硬伤

从本地到云端



从「人→本地Agent→CLI/Skill→环境」变为「人→环境内Agent」

新范式

范式对比：从人驱动到人指挥环境内 Agent

历史 · 给人用的联调	过渡 · 本地 Agent 遥操	未来 · 环境内 Agent ADE
人操作工具，顺序阻塞	人指挥本地 Agent	人只下目标与确认
申请一台，长期占用	Agent 调 CLI / Skill 遥操	Agent 住在环境里完成闭环
一人一环境，一次一任务	环境仍在门外	日抛 / 秒启秒毁 / 语义隔离
人全程 100% 在环上	人仍要编排遥操步骤	人仅在不可逆关口介入

核心转变：人驱动 → 本地遥操 → 人指挥环境内 Agent

04

做成 Agent 友好环境

五大特性 & 核心组件

五大特性

跑得住	7 x 24小时,安全隔离的沙箱
记得住	计算和存储分离,Agent 的「记忆」
管得了	让人和多个agent可以共享、接管、恢复长周期的会话
分得清	出入口流量管控,身份凭证透传
说得通	成本、效果可观测

整体架构


整体架构
客户端进来 · 控制面管会话 · 运行时里住着 Agent
客户端
SDK · CLI · Web
终端 · 移动端
多实例
列表 · 详情 · 拉会话
多会话
创建 · attach 回去
多任务
创建 · 查询 · 停止
控制面
服务端
预热池
模板 · 策略
开放接入
业务方接进来
Runtime 管理
镜像 · 配额
会话创建 / attach
多场景 · 可恢复
会话管理
身份 · 目标 · Provider · 状态 · 会话数据
入口网关
WebSocket · 流式 · HTTP 按会话路由 进得来要对
运行时
环境内
Agent Daemon 会话 · 工作区 · 终端 · 文件
Coding Agent 可插拔
Sandbox · Outside
外部调用 · 凭证隔离
Sandbox · Inside
住进环境 · 联调主路径
出口网关
出向管控 · 凭证注入
上下文
代码 · 依赖 · 运行时
可观测
代码 / Skill 采集
CLI · Skill
环境内可调用
统一文件层
Git · 云盘 · 对象存储
做环境和运行时，不做新的 Coding Agent

核心组件–Agent Sandbox

Agent Outside vs Agent Inside

AS TOOL（外部调用） · DEV Container（环境内执行）

维度	Agent Outside（AS TOOL）	Agent Inside（DEV Container）
场景	job	联调环境 / 云主机
架构	Agent 在外部，经 API 调用沙箱	Agent 在沙箱内部，直接执行
安全性	●●●●● 凭证隔离，攻击面小	●●●○○ 沙箱被攻破 = Agent 被攻破
延迟	●●○○○ 每次交互需网络调用	●●●●● 本地执行，无网络开销
控制性	●●●●● Agent 完全掌控执行流	●●●○○ Agent 受限于沙箱环境
调试	●●○○○ 只能通过 API 观察	●●●●● 可以直接调试
可扩展	●●●●● 可同时调用多种工具	●●●○○ 受限于沙箱内工具
状态管理	●●●●○ Agent 外部维护，清晰	●●●○○ 沙箱内部维护，复杂
适用场景	多租户 SaaS、高安全要求	内部工具、低延迟场景

Outside：高隔离 · 多租户 / 高安全

凭证不进沙箱，代价是网络往返

Inside：低延迟 · 联调主路径

Agent 住进环境，可读运行时、可直接调试

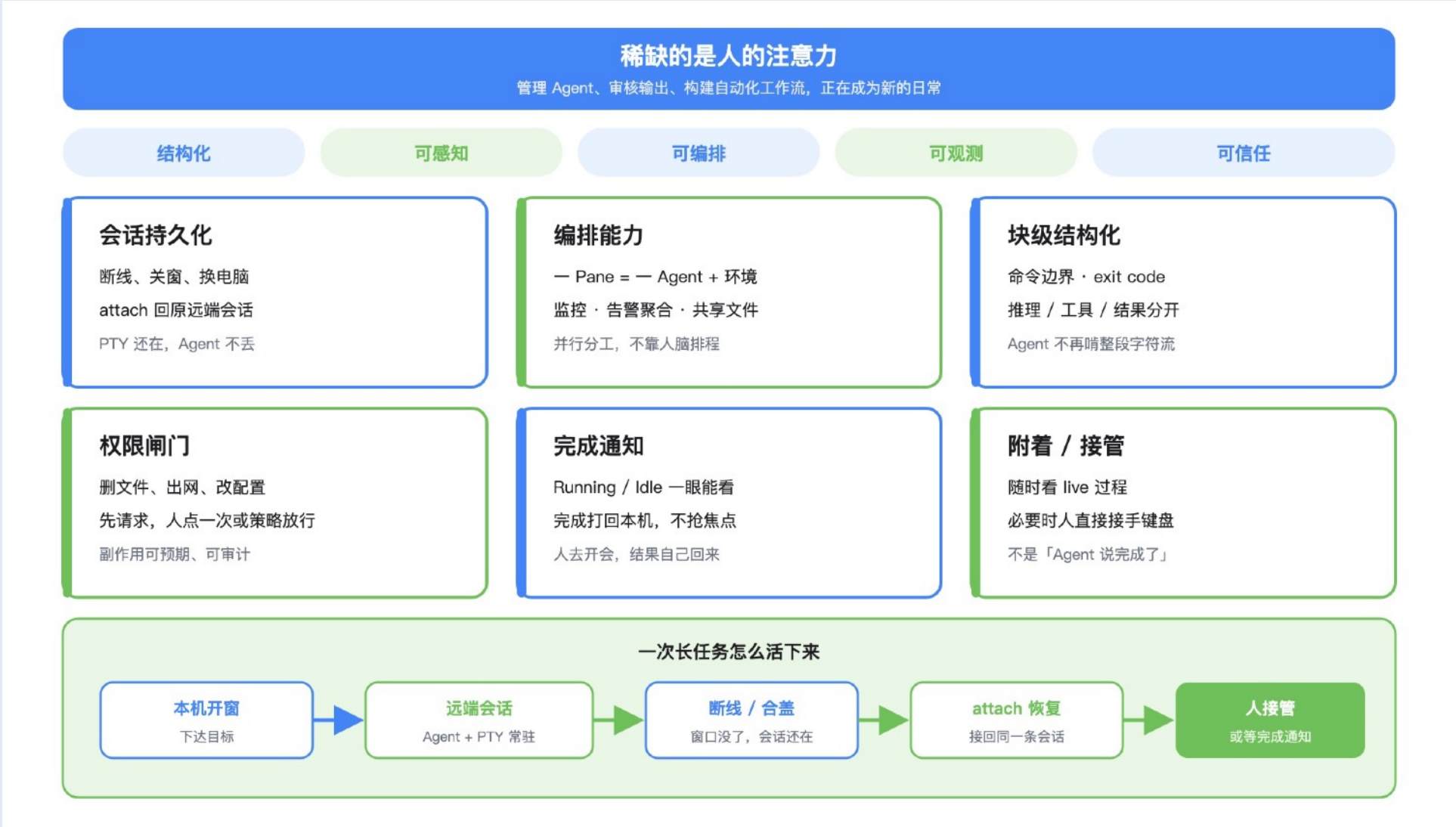
核心组件–Agent Daemon



核心组件–Agent Gateway



核心组件–Agent友好的终端



05

最大的挑战

翻新、验证、协同必须同步

如何翻新

存量环境如何翻新：不是缺环境，是按人设计、Agent 接不上

同一张 DEV / Stable 地图，不能推倒另建一套「给 AI 用的新环境」。

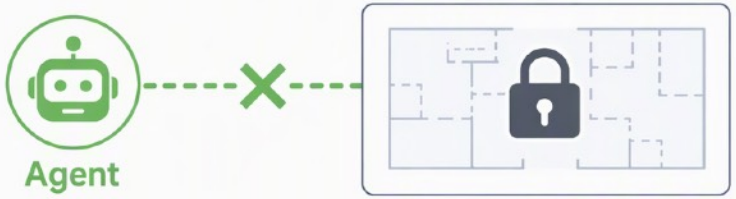
给人用的存量



- 等工位
  开五个控制台
- 问熟手
  眼睛判断「好了没」
- 长期占分组
  慢慢调

短板可以被人补上

Agent 接不上



- 接口面向人
  无可编程入口
- 语义隔离不足
  并行互踩
- 状态不可判定
  无人眼闭环
- 按长占设计
  不能日抛

四件事缺一，坏在第一步就断

假翻新 = 给旧控制台包一层 CLI / Skill，Agent 仍在门外。

真翻新 = 同一张地图改成给 Agent 用的运行时。

技术方案

如何翻新 · 用 Pi 的优势啃存量地图

挑战：机器多 · 内核低 · 主流 Code Agent CLI 装不上 → 解法：Pi Go + DEV 内置 CodeFuse CLI

假翻新：本机装高内核 / Node CLI 遥操旧环境 —— 内核与 glibc 一卡住，整条链断

真翻新：不换一批机器，把「能在低内核上跑的 Agent 工具链」下沉进 DEV

① Go 单二进制

Pi 优势

静态 / 轻量二进制，无 Node 运行时

低 glibc / 旧内核也能拉起

启动快、内存占用低

对翻新：下发到海量 DEV 工位

② 极简 Harness

Pi 优势

默认就 bash + 读写改 · 工具面薄

上下文省 · Token 效率高

不绑重 IDE / 高版本系统能力

对翻新：旧机也能「跑得起来」

③ 可扩展不换核

Pi 优势

Extensions / Skills 适配企业工作流

不必 fork 整套引擎换环境

同一内核服务 IDE / 云端 / 线下

对翻新：同一张地图上长能力

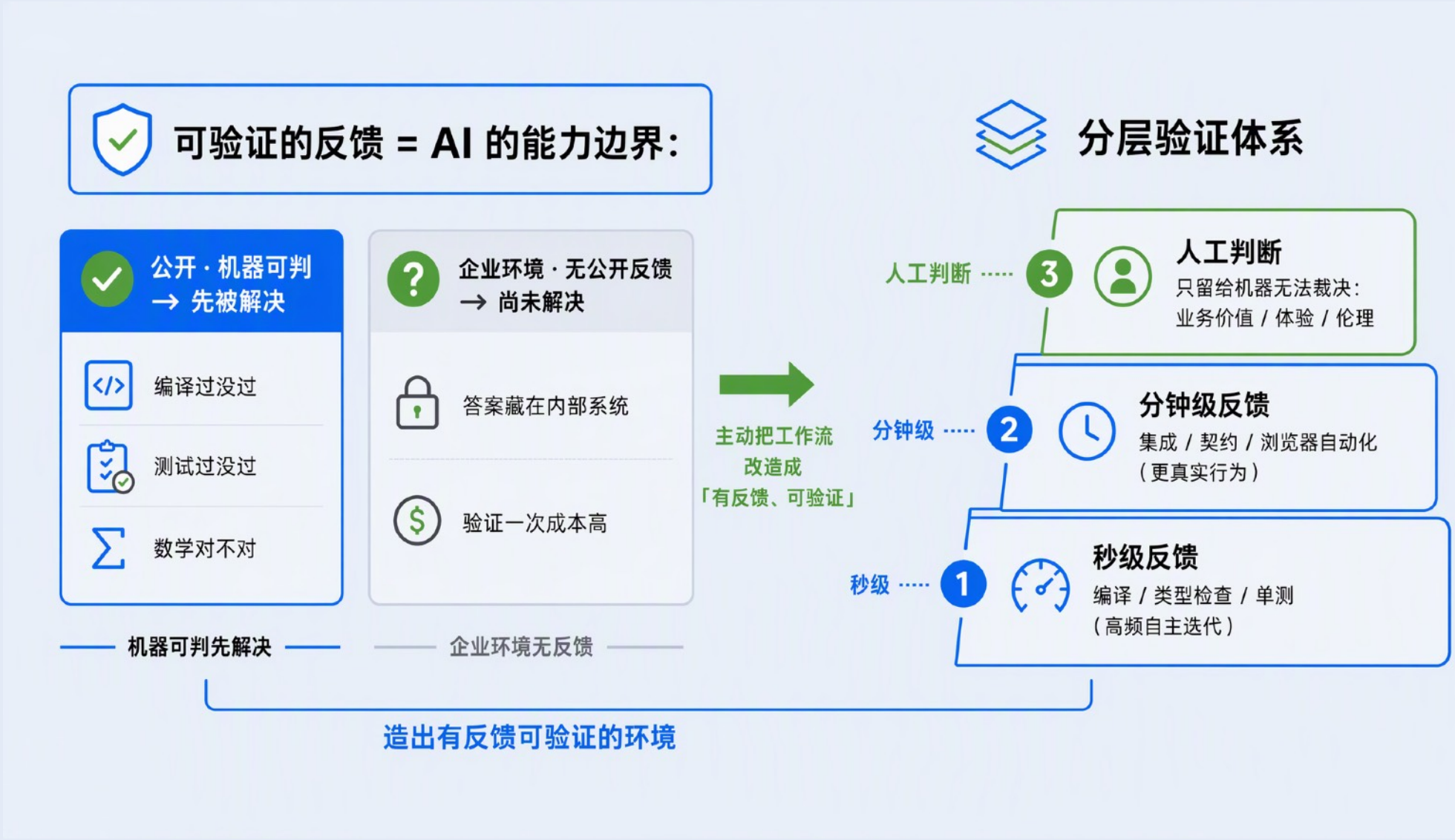
落地组合

① 基于 Pi Go 迭代 Agent 引擎 → 低内核存量机可跑

② DEV 环境内置 CodeFuse CLI → 随工位下发，不依赖开发者本机

结果：Agent 工具链住进环境 —— 翻新的第一步不是换机，是让 CLI 在旧机上活下来

如何验证


可验证的反馈 = AI 的能力边界：
公开 · 机器可判
→ 先被解决
编译过没过
测试过没过
数学对不对
企业环境 · 无公开反馈
→ 尚未解决
答案藏在内部系统
验证一次成本高
机器可判先解决
企业环境无反馈
主动把工作流
改造成
「有反馈、可验证」
分层验证体系
人工判断
3
人工判断
只留给机器无法裁决：
业务价值 / 体验 / 伦理
分钟级
2
分钟级反馈
集成 / 契约 / 浏览器自动化
（更真实行为）
秒级
1
秒级反馈
编译 / 类型检查 / 单测
（高频自主迭代）
造出有反馈可验证的环境

技术方案


验证工作流 — 五层分层架构
任务编排层
eval-plan
交互式参数收集
workflow-param.json
task.json
gate检查
交付层
eval-compound
1a汇总
1b规则回写
1c环境处置
1d中文报告
1e钉钉通知
1f清理dry-run
1g真实清理
验证层
eval-evaluate
5维度打分
ENV_MISMATCH vs CODE_FAILURE
通过/重试/失败判定
重试
执行层
doc-analysis(可选)
代码变更分析
GitNexus+JAR
场景推导
LLM+context
场景规则→资产→
method-info
流量拉取
4a计划→4b采集→
4c构造→4dgate
用例生成
环境部署
linkw-cli
批量验证
6a造数→6bMock
→6c验证
诊断闭环
分类→路由→执行
能力层
环境 CLI
流量 CLI
数据库 CLI
调用 CLI
Mock CLI
配置 CLI
日志 CLI
远端命令 CLI
通知 CLI
文档 CLI

如何协同


指挥面 · 支付确认联调 Tasks: 3 ✓ · 0 ▶ · 0 ⬜
本机开窗 · 远端跑 · 快照 11:26:01 · 仿真 16:00 · ? 帮助
窗 · 环境内 Agent 3 Agent
指挥 本机 working
前端 Agent DEV done ✓
后端 Agent DEV done ✓
运维 Agent Stable done ✓
Stage 指挥 · 本机 working
本机 · 下达目标 / 接管
— 指挥面 · 本机开窗 · 远端跑 —
> 把支付确认页这条联调跑通。前端写 A，后端读 A 写 B，运维读 B。人不传话。
› 已挂三个窗，开始看共享文件，而不是群聊。
› 三个窗已挂上：DEV 前端 / DEV 后端 / Stable 运维
› 三个窗都交回了。人没有当过中间人。
⚠ 人要看的 4 项
done 前端 Agent
#21 完成（产物已在共享文件 A）
Enter 看交接结果
done 后端 Agent
#22 完成（报告已在共享文件 B）
Enter 看交接结果
done 运维 Agent
#23 完成（Stable 对照完成）
Enter 看交接结果
done 指挥
一条目标跑完。交接全在共享文件 A / B。
Enter 看交接结果
任务 3 项
#21 ✓ 改确认页交互，产物写到共享文件 A 前端
#22 ✓ 读共享文件 A，联调，写出报告 B 后端
#23 ✓ 读报告 B，在 Stable 对照复跑 运维
消息 2 新
· 14:27 运维 Agent → 后端 Agent
读取共享文件 B ← shared/confirm-report.md
● 14:24 后端 Agent
写下共享文件 B → shared/confirm-report.md（联调报告：假绿，riskToken 为空）
· 14:14 后端 Agent → 前端 Agent
读取共享文件 A ← shared/confirm-page.md
● 14:11 前端 Agent
产物 2/2 就绪
A shared/confirm-page.md
写入 前端 Agent · 已读 后端 Agent · 确认页产物：交互 diff + 页面状态
B shared/confirm-report.md
写入 后端 Agent · 已读 运维 Agent · 联调报告：假绿，riskToken 为空
协同靠共享文件，别让人当中间人
STAGE Tab 切窗 1–4 直达 a 人要看的 y/n 确认 s 下达一句 ? 帮助

如何协同


协同场：人通过指挥面，指挥多环境里的 Agent
01 指挥面
人
下达目标
可附着 · 可接管
可恢复
执行面 · 本机开窗
Pane · 前端 Agent
DEV 环境
Pane · 后端 Agent
DEV 环境
Pane · 运维 Agent
Stable 环境
本机开窗 · 远端跑 · 下令到环境内 Agent
指挥面
=
执行面
+ 共享文件
+ 基础设施
02 共享文件
多 Agent 读写同一批产物 · 不全量复制进对话
任务
产物
报告
代码
文档
共享文件交接
03 基础设施
跑得住 · 记得住 · 管得了 · 可挂载
运行时
产物存储
网关
云盘
上下文不靠人
复制粘贴
用共享文件
接着跑

06

总结&规划

总结

总结

Poolab · 业务效果

业务效果

云主机日常工位
打开就能干活，不是反复申请机器

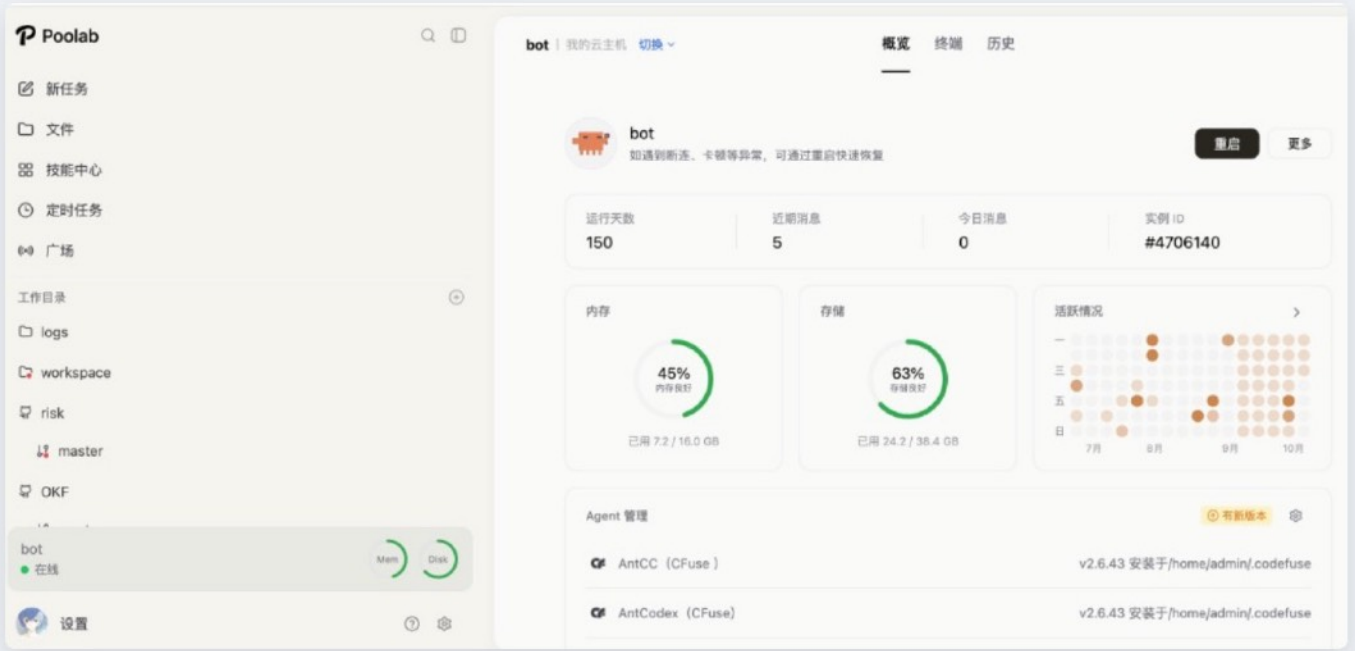
IM 触达协作
钉钉机器人把能力送进群聊与对话

Custom Agent
团队按场景自建专属助理

场景上翻
线下联调 / 长程 harness 同一张地图

人与 Agent 都能用
可跑 · 可验 · 可日抛

我的云主机



编码不是瓶颈，环境才是 · 别再给 Agent 一个毛坯房

总结

总结

Poolab · 业务效果

业务效果

云主机日常工位
打开就能干活，不是反复申请机器

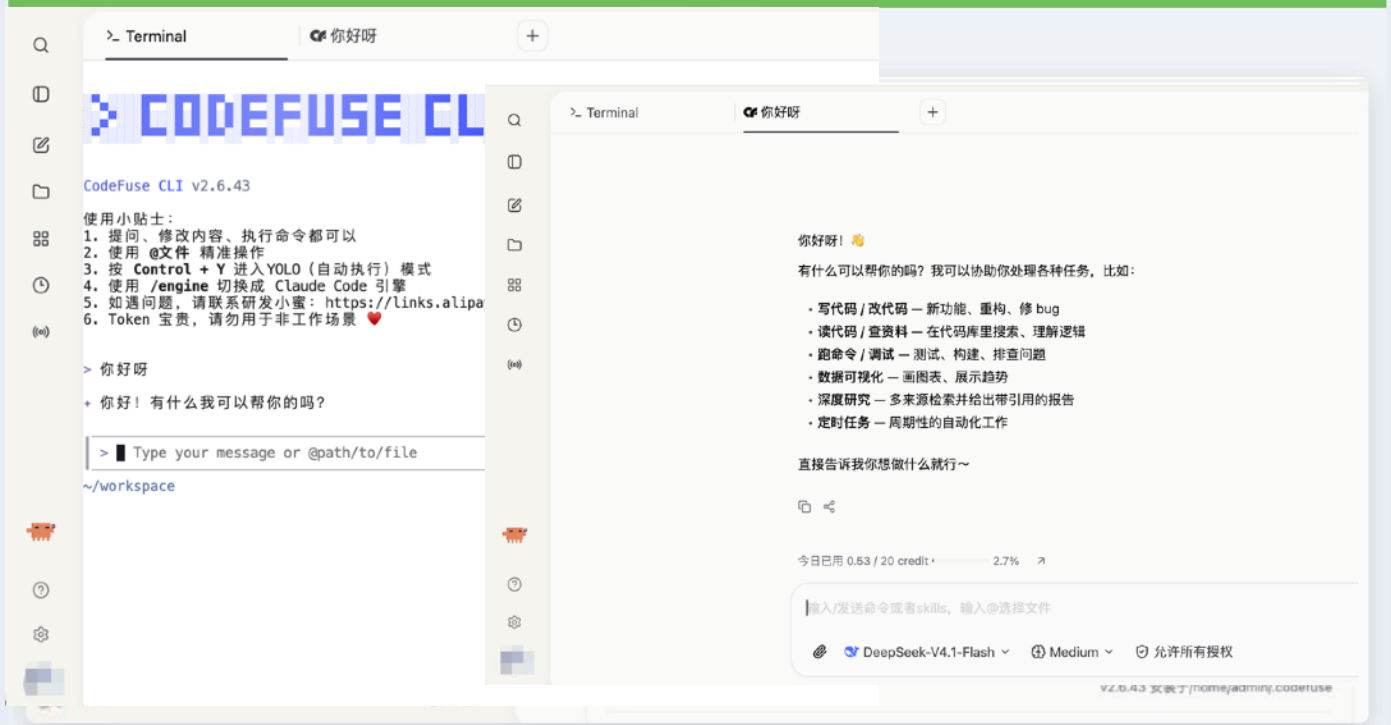
IM 触达协作
钉钉机器人把能力送进群聊与对话

Custom Agent
团队按场景自建专属助理

场景上翻
线下联调 / 长程 harness 同一张地图

人与 Agent 都能用
可跑 · 可验 · 可日抛

我的云主机



编码不是瓶颈，环境才是 · 别再给 Agent 一个毛坯房

总结

总结

Poolab · 业务效果

业务效果

云主机日常工位
打开就能干活，不是反复申请机器

IM 触达协作
钉钉机器人把能力送进群聊与对话

Custom Agent
团队按场景自建专属助理

场景上翻
线下联调 / 长程 harness 同一张地图

人与 Agent 都能用
可跑 · 可验 · 可日抛

我的云主机

在钉钉机器人里使用 Poolab

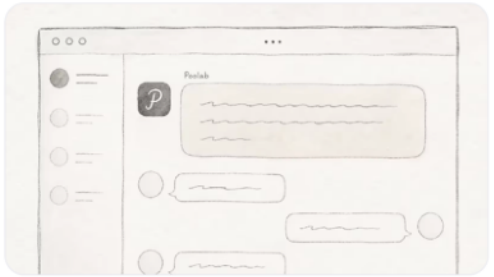
钉钉消息即是任务，随时随地发起

官方 Poolab 钉钉机器人 更稳定 | 自定义钉机器人



单聊，自己用

蚂蚁钉里搜索 Poolab，直接发起对话
对他输入 `/help` 可查看使用指南



钉钉群，分享给群成员共用

需要先在钉钉群中完成绑定
绑定后，群成员都可以 @Poolab 并和你的云主机对话

编码不是瓶颈，环境才是 · 别再给 Agent 一个毛坯房

未来规划

未来规划

基于云主机的个人助理

从「工具」到「同事」· 助理住在云主机里

合上本机盖子，它还在线上班 · 默认云端执行，本机只指挥

原则 · 云端优先

本机扛不住 Agent

隔离 · 并发 · 一致性 · 7×24

不可信代码不能裸奔本机

架构取舍

薄客户端 · 厚服务端

写、跑、联默认在云端 · 本机只做对话 / 审批 / 附着

云端是主路径 · 本机是旁路

形态 · 个人助理

Server → Computer

每人一台长期在线云主机

文件 · 浏览器 · 桌面都在云端

两种生命周期并存

短程日抛舱 · 一次性验证

长程云主机 · 7×24 托付 · 空闲可挂起

沙箱是舱 · 云主机才是工位

下一步：给每人一台「个人助理的电脑」

云端优先执行 · 长程常驻工位 · 系统级信任边界

本机做人该做的事 · 云主机给助理上班

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师